```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

void bubbleSort(int arr[], int n)
{
    int i, j, temp;

    for (i = 0; i < n - 1; i++)
    {
        for (j = 0; j < n - i - 1; j++)
        {
            if (arr[j] > arr[j + 1])
            {
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

void display(int arr[], int n)
{
    int i;

    for (i = 0; i < n; i++)
        printf("%d ", arr[i]);

    printf("\n");
}

void demonstrateZombie()
{
    pid_t pid;

    printf("\n--- ZOMBIE PROCESS ---\n");

    pid = fork();

    if (pid == 0)
    {
```

```
        printf("Child process terminated.\n");
        exit(0);
    }
    else if (pid > 0)
    {
        printf("Parent is sleeping. Child becomes zombie.\n");
        sleep(3);

        wait(NULL);

        printf("Parent called wait(). Zombie removed.\n");
    }
}

void demonstrateOrphan()
{
    pid_t pid;

    printf("\n--- ORPHAN PROCESS ---\n");

    pid = fork();

    if (pid == 0)
    {
        printf("Child PID: %d\n", getpid());
        printf("Old Parent PID: %d\n", getppid());

        sleep(3);

        printf("New Parent PID: %d\n", getppid());
        printf("Child is now an orphan process.\n");
    }
    else if (pid > 0)
    {
        printf("Parent is terminating.\n");
        exit(0);
    }
}

int main()
{
    int n, i;
    int arr[100], childArr[100];
```

```
pid_t pid;

printf("Enter number of integers: ");
scanf("%d", &n);

printf("Enter integers:\n");

for (i = 0; i < n; i++)
{
   scanf("%d", &arr[i]);
   childArr[i] = arr[i];
}

pid = fork();

if (pid < 0)
{
   printf("Fork failed!\n");
   return 1;
}

else if (pid == 0)
{
   printf("\n--- CHILD PROCESS ---\n");

   printf("Child PID: %d\n", getpid());

   bubbleSort(childArr, n);

   printf("Sorted array (Child): ");
   display(childArr, n);

   exit(0);
}

else
{
   printf("\n--- PARENT PROCESS ---\n");

   printf("Parent PID: %d\n", getpid());

   bubbleSort(arr, n);

   printf("Sorted array (Parent): ");
```

```
        display(arr, n);

        wait(NULL);

        printf("Parent: Child process completed.\n");
    }

    demonstrateZombie();

    demonstrateOrphan();

    return 0;
}
```